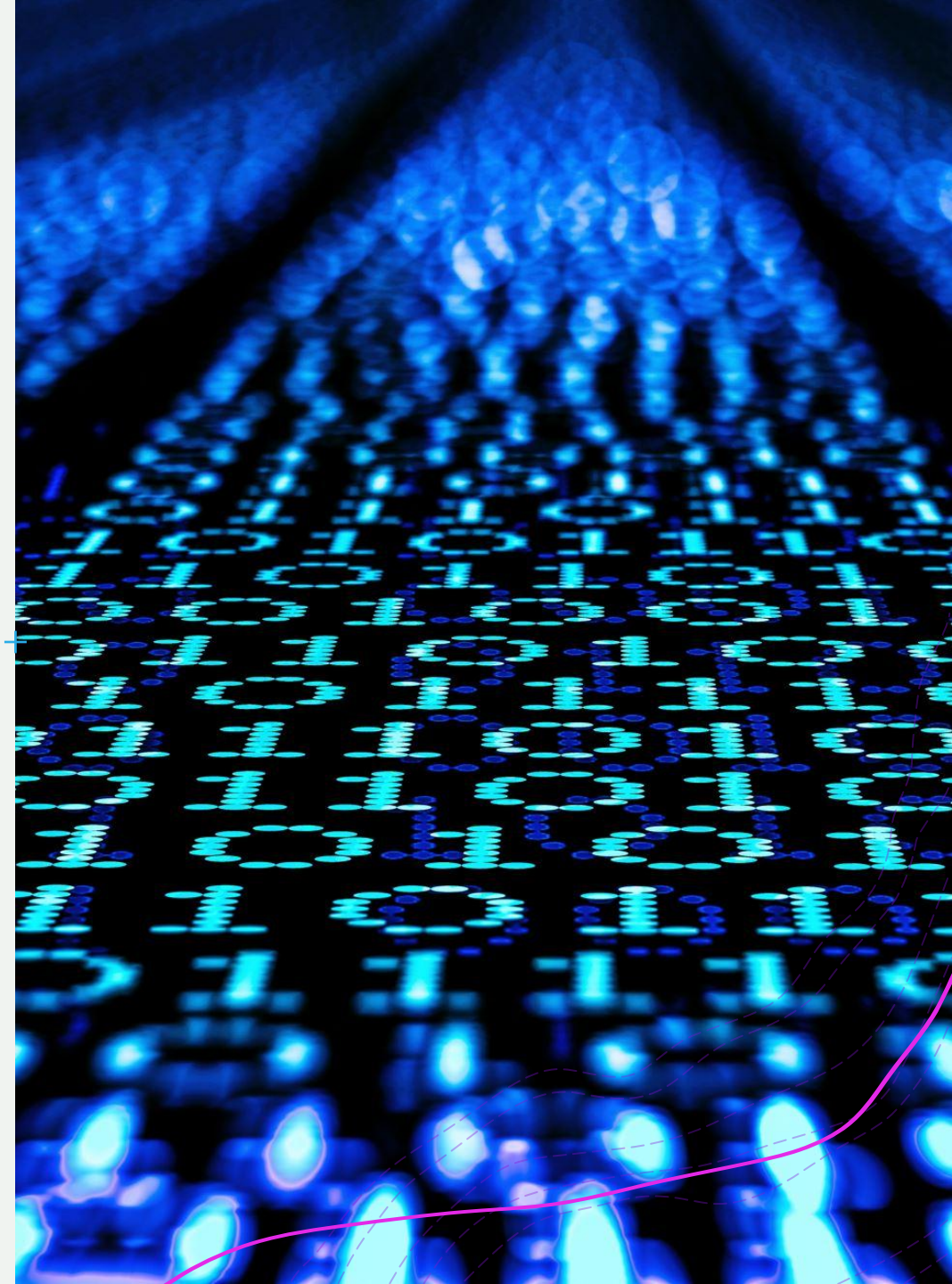
Shmuel BITAN
Protections contre l'exploitation d'une pile

RSX 112



Qu’est-ce que le Stack ?

Le **stack** (pile) est une zone de mémoire temporaire utilisée par les programmes.

•Il gère :

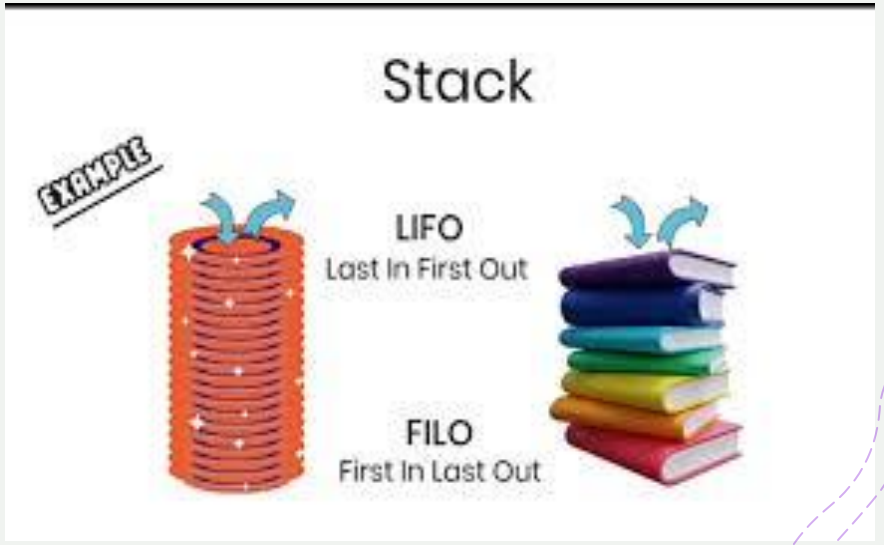
1.Les appels de fonctions (adresses de retour).

2.Les variables locales des fonctions

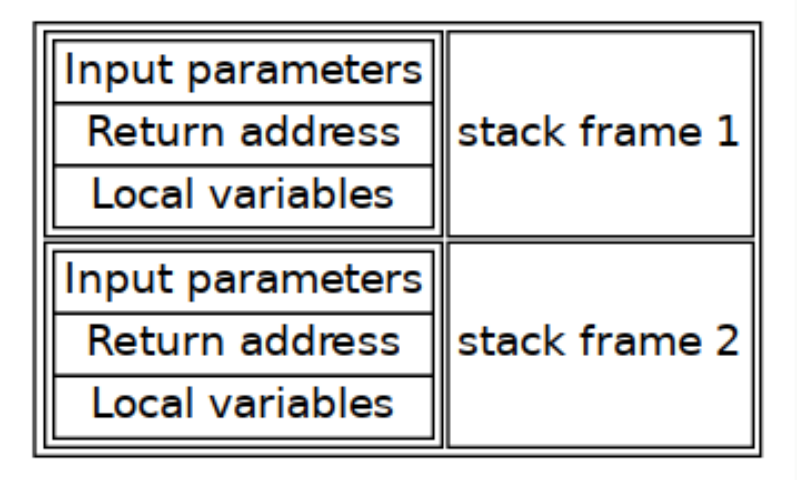
3.Les arguments passés aux fonctions

•Fonctionne sur le principe **LIFO** (*Last In, First Out*)

•Les données récentes sont empilées et retirées en premier



Fonctionnement et criticité du Stack



Lorsqu'une fonction est appelée, un cadre de stack (*stack frame*) est créé.

Les données suivantes sont empilées :

- Arguments
- Variables locales
- Adresse de retour

Lorsque la fonction se termine, son cadre est dépilé, et le programme reprend à l'adresse de retour.

Le stack est critique parce qu'il contient des données critiques :

- Variables temporaires
- Adresse de retour

Une mauvaise gestion du stack peut **exposer ces données** à des attaques.

Qu’est-ce qu’un Stack Overflow ?

Un **stack overflow** se produit lorsqu’un buffer dépasse ses limites et écrase d’autres données dans le stack. L’**adresse de retour** est la cible principale.

Objectif de l’attaquant :
•Rediriger l’exécution du programme vers un code malveillant.

Quelles sont les étapes d'une attaque par Stack Overflow ?

- Trouver l’**offset** : Identifier la position de l’adresse de retour dans le stack
- Injecter un **shellcode** dans la mémoire
- Écraser l’adresse de retour avec une adresse pointant vers le shellcode

Conséquences d'un Stack Overflow

Permet à l'attaquant de :

- Exécuter un shell interactif
- Élever ses privilèges
- Installer un logiciel malveillant

Impact :

- Perte de contrôle du système
- Accès non autorisé aux données sensibles

Contexte et origine du bit NX/XD

Contexte historique

Les attaques exploitant des débordements de buffer (comme les stack overflows) sont apparues dès les années 1980.

Une attaque classique consistait à injecter un shellcode dans une zone mémoire (comme le stack) et à l'exécuter.

Problème fondamental

Dans les architectures classiques, toutes les zones mémoire étaient souvent à la fois :

- Lues (Read).
- Écrites (Write).
- Exécutées (Execute).

Cela permettait à un attaquant d'injecter du code malveillant dans une zone non prévue pour l'exécution (ex. : le stack ou le heap).

Introduction du bit NX/ND

Développé par AMD en 2004 et intégré dans ses processeurs x86-64.

Intel l'a adopté sous le nom XD (eXecute Disable).

Objectif : Séparer les zones mémoire utilisées pour les données des zones utilisées pour le code.

Comment fonctionne le bit NX/XD ?

Principe	• Chaque page mémoire a un attribut **NX (No-eXecute)**. • Si une page est marquée **NX**, elle ne peut pas exécuter de code, même si elle est accessible en lecture ou en écriture.
Application concrète	• **Stack et Heap** : Généralement marqués comme NX, empêchant l'exécution de shellcodes injectés. • **Code exécutable** : Les segments de code restent marqués comme exécutables.
Protection	• Bloque les attaques exploitant des débordements pour exécuter du code injecté. • Provoque une exception (segmentation fault) si un programme tente d'exécuter une page marquée NX.
Limitations	• Inefficace contre des techniques comme le **ROP (Return-Oriented Programming)**, qui réutilisent du code existant.

Origine et principe de l'ASLR

Contexte historique

- Les attaques de type débordement de mémoire exploitent des **adresses fixes** en mémoire pour détourner le programme.
- Avant l’ASLR, les zones critiques comme le **stack**, le **heap**, ou les **bibliothèques dynamiques** étaient chargées aux mêmes adresses à chaque exécution.

Principe de l’ASLR

- L'ASLR (Address Space Layout Randomization) a été introduit pour **rendre imprévisible l’agencement de la mémoire**.
- À chaque exécution du programme, les segments mémoire sont déplacés à des adresses aléatoires :
  - Stack.
  - Heap.
  - Bibliothèques dynamiques (shared libraries).
  - Exécutable lui-même.

Avantage clé

- L'attaquant ne peut plus prédire les adresses mémoire, ce qui rend plus difficile l’exploitation de vulnérabilités.

Fonctionnement de l'ASLR et ses limites

Fonctionnement technique

Lorsqu'un programme démarre, le système d'exploitation :

- Randomise l'adresse du **stack**.
- Randomise l'adresse du **heap**.
- Charge les **bibliothèques dynamiques** et l'exécutable lui-même à des adresses aléatoires.

Les adresses changent à chaque exécution du programme.

Protection offerte

Les attaques nécessitant des adresses fixes (par exemple, pour exécuter un shellcode ou une attaque ROP) sont beaucoup plus difficiles à réaliser.

Limitations de l'ASLR

Attaques de fuite d'informations :

Si un attaquant parvient à lire une adresse mémoire, il peut reconstituer l'agencement.

Compatibilité partielle :

Certains programmes anciens ou matériels limités n'activent pas l'ASLR.

Failles partielles :

L'ASLR peut être contourné si la randomisation est faible (par exemple, peu d'entropie).

Qu’est-ce qu’un Canary ?

Taille et structure :

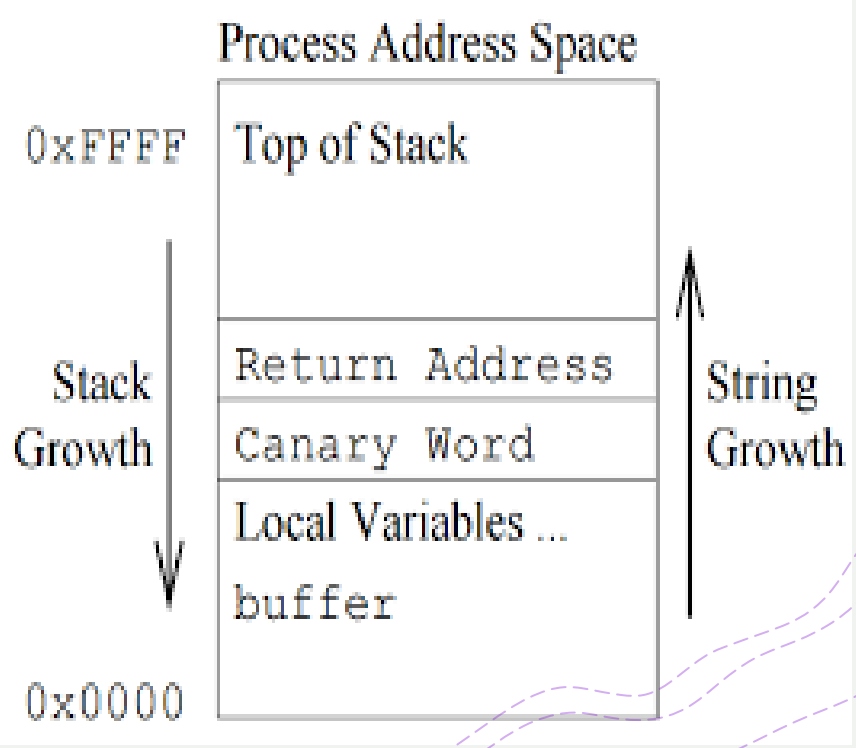
1. Un canary est une valeur **aléatoire de 32 bits**, soit **4 bytes**.
2. Il est généré à chaque **appel de fonction** et inséré dans le stack entre les **variables locales** et l’**adresse de retour**.

Rôle du canary :

1. Si un débordement de buffer essaie d’écraser l’adresse de retour, il modifiera le canary.
2. Avant de retourner, le programme compare le canary à sa valeur initiale :
   1. **Intact ?** Le programme continue normalement.
   2. **Modifié ?** Une alerte est déclenchée et le programme s’arrête.

Efficacité :

1. Théoriquement, il y a **2^{32} possibilités** pour un canary (4 294 967 296 valeurs), ce qui rend un test exhaustif impossible en une seule tentative.



Fonctionnement détaillé des canaries

Mise en place d'un canary

- À l'entrée d'une fonction, le compilateur génère un **canary aléatoire** et le place dans le stack.
- Les canaries sont généralement de 4 ou 8 octets et peuvent inclure des valeurs spécifiques pour augmenter leur efficacité.

Types de canaries

- **Canaries aléatoires** :
  - Générés dynamiquement à chaque exécution.
- **Canaries terminaux** :
  - Incluent des octets de fin de chaîne (\0) pour bloquer les fonctions comme strcpy ou gets, qui s'arrêtent sur ce caractère.
- **Canaries XOR** :
  - Générés en combinant l'adresse de retour et une clé secrète pour renforcer leur unicité.

Vérification du canary

- Juste avant de retourner, le programme compare la valeur actuelle du canary à celle initialement enregistrée.
- Si elles diffèrent, un mécanisme d'urgence est déclenché (souvent un arrêt immédiat du programme).

Les Limites des Stack Canaries : Exploitation via Forks

Problème de génération aléatoire :

Les canaries sont générés à chaque appel de fonction, offrant **2^{32} possibilités** pour chaque tentative.

Cependant, dans un programme utilisant un **fork()**, les processus fils héritent du **même canary** que le processus père.

Rôle du fork dans l'exploitation :

L'attaquant peut lancer plusieurs processus fils via des forks et tester les bytes du canary un par un.

Chaque byte ayant 256 possibilités, il peut reconstruire le canary en :

- **256 x 4 = 1024 tests maximum**.

Exploitation pratique :

L'attaquant exploite un programme vulnérable qui retourne une réponse observable (ex. : crash ou log) pour chaque test incorrect.

En observant ces réponses, il peut progressivement deviner les bytes du canary.

Conclusion :

Bien que les canaries soient robustes, leur sécurité peut être contournée dans des environnements spécifiques comme les programmes à forks.

Introduction à SafeSEH

Contexte et problème :

- Les exceptions structurées (SEH) sont des mécanismes utilisés par Windows pour gérer les erreurs et exceptions pendant l'exécution d'un programme.
- Les attaquants peuvent exploiter des débordements de buffer pour **remplacer les gestionnaires d'exceptions** par des adresses malveillantes.
- Objectif de l'attaquant : détourner le flux d'exécution du programme via un faux gestionnaire d'exceptions.

SafeSEH : Une réponse de sécurité :

- Introduit par Microsoft pour **valider les gestionnaires d'exceptions** avant leur exécution.
- Fonctionne uniquement pour les modules (DLLs ou exécutables) compilés avec cette fonctionnalité.
- Si un gestionnaire d'exception n'est pas déclaré dans une table valide, l'exécution est bloquée.

Avantages :

- Empêche l'exécution de gestionnaires d'exceptions injectés ou non autorisés.
- Protège contre les exploits basés sur la corruption des structures SEH.

Fonctionnement et limites de SafeSEH

Fonctionnement technique :

1. Lorsqu'un gestionnaire d'exception est appelé, Windows vérifie s'il est présent dans une **table des gestionnaires autorisés**, incluse dans le binaire au moment de la compilation.
2. Si le gestionnaire est valide, il s'exécute normalement.
3. Sinon, une exception est levée et le programme est bloqué.

Compatibilité :

1. Fonctionne uniquement pour les binaires compilés avec **SafeSEH** activé.
2. Non applicable aux modules qui n'utilisent pas ce mécanisme (ex. : certains modules tiers ou anciens).

Limitations :

1. Inefficace contre des techniques comme **SEH Overwrite Bypass**, où l'attaquant exploite un module non protégé par SafeSEH.
2. Ne protège pas contre les vulnérabilités dans d'autres parties du programme.

Introduction à PaX/Grsecurity

Contexte :

Les attaques exploitant des vulnérabilités mémoire (stack overflow, heap overflow) sont parmi les plus courantes.

Les protections natives comme DEP et ASLR ne couvrent pas tous les cas.

PaX :

Une série de correctifs pour le noyau Linux introduisant des protections avancées contre les attaques mémoire.

Protége notamment :

- Le **stack** (dépassements).
- Les **zones exécutables** (prévention de l'exécution dans des zones non prévues).

Grsecurity :

Une suite de sécurité intégrant PaX et ajoutant :

- Des mécanismes de contrôle d'accès renforcés (RBAC).
- Une meilleure journalisation.
- Une défense proactive contre les comportements suspects.

Objectif commun :

Rendre l'exploitation des vulnérabilités beaucoup plus difficile en augmentant la complexité des attaques.

Fonctionnalités et limites de PaX/Grsecurity

Fonctionnalités principales de PaX :

- **DEP amélioré** : Empêche l'exécution de code dans des zones comme le stack ou le heap.
- **ASLR avancé** : Randomise non seulement les segments standard (stack, heap), mais aussi les adresses des fonctions.
- **Protection des segments** : Détecte et bloque les dépassements de buffers avant qu'ils ne soient exploités.

Grsecurity :

- **RBAC (Role-Based Access Control)** : Limite les droits des processus et des utilisateurs en fonction de leur rôle.
- **Journalisation avancée** : Permet de détecter les comportements suspects en temps réel.
- **Hardening général** : Supprime les fonctionnalités inutiles dans le noyau pour réduire les surfaces d'attaque.

Limites :

- Non intégré par défaut dans les noyaux Linux : nécessite une installation manuelle.
- **Impact sur les performances** : Certaines fonctionnalités ajoutent une charge, notamment dans les environnements à ressources limitées.

Compatibilité avec les applications existantes

NX/XD (No-Execute/eXecute-Disable) :

Compatibilité :

- Largement supporté par la majorité des applications modernes.
- Certaines anciennes applications écrivant du code auto-modifiable ou exécutant des données dans le stack/heap peuvent poser problème.

Solution : Activer NX/ND seulement pour des segments spécifiques ou désactiver NX pour des applications particulières.

ASLR (Address Space Layout Randomization) :

Compatibilité :

- Bien supporté par les applications modernes.
- Les anciens programmes utilisant des adresses statiques ou des bibliothèques non compatibles ASLR peuvent échouer.

Solution : Maintenir des bibliothèques compatibles ASLR.

Canaries :

Compatibilité :

- Généralement transparent pour les développeurs et utilisateurs.
- Peut provoquer des problèmes dans certains cas de code mal optimisé ou non sécurisé (ex. : manipulation directe du stack).

SafeSEH :

Compatibilité :

- Fonctionne uniquement avec les binaires compilés avec SafeSEH activé.
- Les anciens binaires ou les modules tiers sans SafeSEH restent vulnérables.

Solution : Recompiler les binaires ou remplacer les modules non compatibles.

PaX/Grsecurity :

Compatibilité :

- Peut poser des problèmes avec des applications nécessitant un accès direct à la mémoire ou exécutant du code dans le heap.
- Certaines fonctionnalités, comme RBAC, peuvent exiger des ajustements pour les permissions.

Solution : Ajuster les règles RBAC ou désactiver temporairement les protections pour certaines applications.

Compatibilité avec les applications existantes

NX/ND (No-Execute/eXecute-Disable) :

- **Impact** :
  - Minimal, car il repose sur des drapeaux matériels (supportés par les processeurs modernes).
  - Aucun impact significatif sur la vitesse d'exécution des programmes.

ASLR (Address Space Layout Randomization) :

- **Impact** :
  - Léger sur le démarrage des programmes, car les adresses mémoire doivent être randomisées.
  - Pratiquement négligeable pour la plupart des applications en cours d'exécution.

Canaries :

- **Impact** :
  - Négligeable, car les opérations liées au canary (génération et vérification) sont rapides et ajoutent peu de surcharge.

SafeSEH :

- **Impact** :
  - Léger, car les validations des gestionnaires d'exceptions ajoutent une surcharge minime.
  - Aucun impact notable sur les performances globales des applications.

PaX/Grsecurity :

- **Impact** :
  - Peut être plus significatif :
  - Les protections comme ASLR renforcé ou DEP amélioré augmentent la complexité des allocations mémoire.
  - RBAC et les fonctionnalités de journalisation peuvent affecter légèrement les performances dans les environnements à charge élevée.
- Nécessite une configuration optimisée pour limiter les surcharges.

Bibiographie

Try hack me comprendre les buffers et mise en pratique

- https://tryhackme.com/r/room/bof1

La théorie derrière les stacks et le buffer

- https://youtu.be/1S0aBV-Waeo?si=0uD09tXBuNnRyqbI

Le fonctionnement des canaries

- https://youtu.be/N7kGd76evsM?si=LIJHE4t88TaqwAjD

ASLR

- https://www.wallarm.com/what/what-is-aslr-address-space-layout-randomization

Bit NX/XD

- https://youtu.be/5ASe_F-9zx8?si=TwhGJVcHWIwfnlYz

SafeSEH

- https://medium.com/@boutnaru/the-windows-security-journey-safeseh-safe-structured-exception-handling-835b6b938468

Pax/Grsecurity

- https://www.debian.org/doc/manuals/securing-debian-manual/ch04s15.en.html